# Seguridad de MongoDB 8.0

## Unidad 1 — Identificarse no significa tener permiso para todo

### Un ejemplo fuera de la informática

Un empleado entra en un hospital con su tarjeta. La tarjeta demuestra **quién es**, pero no le permite consultar cualquier historia clínica ni modificar todos los registros. Para esas operaciones necesita permisos concretos.

La misma separación existe en MongoDB:

- **Autenticación:** ¿quién eres? Por ejemplo, usuario y contraseña mediante SCRAM.
- **Autorización:** ¿qué puedes hacer? Por ejemplo, consultar documentos sin poder modificarlos.
- **Red:** ¿puede llegar el cliente al servidor?
- **TLS:** ¿viaja protegida la información entre cliente y servidor?

Son preguntas distintas. Tener una contraseña válida no concede todos los permisos, y TLS no convierte a alguien en administrador.

### Nuestro laboratorio protegido

El proyecto utiliza un servidor `mongo-seguridad` separado del servidor de índices. Su base de datos de ejemplo es `curso_seguridad` y contiene una colección `secretos` con un documento completamente ficticio. Hay tres identidades de práctica:

| Cuenta | Finalidad | Permisos |
|---|---|---|
| `adminlab` | Preparar el entorno | Administración |
| `lectorlab` | Consultar un registro | Rol `read` |
| `escritorlab` | Añadir registros | Rol `readWrite` |

El usuario lector puede recuperar un documento, pero debe recibir un error al intentar insertarlo. **En este caso, el error es una prueba satisfactoria:** demuestra que MongoDB aplica una restricción.

### Cómo funcionan los roles

MongoDB utiliza control de acceso basado en roles, conocido como **RBAC**. Un rol reúne permisos; a cada cuenta se le asignan roles adecuados a sus tareas.

El principio de **mínimo privilegio** significa conceder solo lo imprescindible. Si una aplicación necesita mostrar informes, una cuenta administrativa tendría demasiados privilegios. Ante un error de autorización, no deberíamos responder concediendo `root` indiscriminadamente: primero hay que comprobar la operación necesaria.

## ¿Qué es la base de autenticación?

Una cuenta se crea asociada a una base de datos de autenticación. Por eso en una conexión aparece `--authenticationDatabase` o el concepto `authSource`. Si una cuenta está creada en `curso_seguridad`, autenticarla contra otra base puede fallar aunque el nombre y la contraseña sean correctos.

## Dos errores de causa distinta

**Caso A:** alguien intenta leer sin haberse autenticado. El servidor rechaza la consulta.

**Caso B:** una cuenta válida con rol `read` intenta insertar. El servidor también rechaza la operación, pero ahora **la identidad sí es válida**: falta autorización para escribir.

La práctica guiada reproduce ambos casos. Aprenderás a distinguirlos por su causa, no solo por el mensaje de error.

## Protección del canal y los datos

TLS crea un canal cifrado y permite validar la identidad del servidor con certificados. La autenticación X.509 puede identificar clientes mediante certificados cuando está configurada. Ni TLS ni X.509 sustituyen la comprobación de roles. El cifrado en reposo es otra cuestión: protege información almacenada frente a determinadas amenazas y sus mecanismos dependen del producto y de la infraestructura.

En MongoDB 8.0, la integración LDAP está obsoleta; las capacidades empresariales que no están en MongoDB Community no se presentarán como si pudieran probarse en nuestro contenedor gratuito.

## Comprueba tu comprensión

1. ¿Qué control actúa si alguien introduce bien su contraseña, pero no puede insertar?
2. ¿Por qué no deberíamos utilizar `adminlab` como cuenta de una aplicación que solo lee?
3. ¿Por qué un error `Unauthorized` puede ser el resultado correcto de una práctica?
4. ¿En qué se diferencia proteger la conexión con TLS de asignar el rol `read`?

**Respuesta a la pregunta 1:** la autorización. La autenticación identificó a la cuenta, pero el rol no permite esa acción.

---

# Unidad 2. La seguridad comienza antes de introducir la contraseña

## 2.1. Qué estamos protegiendo

Cuando hablamos de seguridad de una base de datos no queremos decir únicamente que la aplicación tenga una pantalla de inicio de sesión. Una base contiene información y soporta operaciones: consultar clientes, registrar pedidos, borrar registros, exportar informes o administrar usuarios. Cada una de esas acciones puede tener consecuencias diferentes.

Antes de configurar controles, enumera tres preguntas:

- **Qué datos existen.** No tiene la misma sensibilidad una lista de códigos de prueba que los datos personales o financieros de clientes reales.
- **Quién necesita operar.** Un proceso que registra ventas, una persona que consulta informes y un administrador no requieren los mismos poderes.
- **Qué puede salir mal.** Acceso desde una red no deseada, robo de credenciales, modificación accidental, error de configuración o pérdida de una copia de seguridad.

Si no hacemos estas preguntas, podemos configurar una contraseña segura y aun así permitir que cualquier cuenta elimine una colección completa.

## 2.2. El puerto no equivale al permiso

MongoDB escucha conexiones a través de una red. En Docker Compose publicamos puertos para que un cliente del ordenador pueda comunicarse con el contenedor.

En el laboratorio, las asignaciones son:

| Servicio | Puerto del ordenador | Uso |
|---|---|---|
| `mongo-indices` | `127.0.0.1:27017` | Ejercicios con datos ficticios |
| `mongo-seguridad` | `127.0.0.1:27018` | Autenticación y permisos |
| `mongo-tls` | `127.0.0.1:27019` | TLS y certificados |

`127.0.0.1` significa el equipo local o *loopback*. Limita la exposición de los puertos publicados en el host, pero **no constituye por sí mismo una política de autenticación**. Los contenedores conectados a la red de Docker pueden tener rutas internas diferentes; por eso también necesitamos diseñar controles en el servidor.

Una analogía: cerrar el acceso exterior al edificio no significa que cualquier persona dentro de él deba tener las llaves de todas las habitaciones.

## 2.3. Autenticación y autorización: el orden de las preguntas

Supongamos que el empleado `lectorlab` quiere consultar un documento.

1. Establece una conexión con el servidor.
2. Presenta credenciales para que MongoDB conozca su identidad.
3. El servidor determina qué roles tiene asociados.
4. Cuando solicita `find` sobre una colección, el servidor comprueba si sus privilegios permiten esa acción.
5. Si no tiene permiso, la operación se deniega; **no deberíamos modificar los roles solo para ocultar el error de clase**.

La autenticación no es algo que debamos repetir con cada documento visible al usuario: el servidor mantiene información de la identidad dentro de la sesión de conexión. La autorización se aplica a las operaciones que esa sesión intenta realizar.

## 2.4. Separar cuentas es separar responsabilidades

En una organización real puede haber una cuenta para una aplicación web, otra para procesos de integración, otra para copias de seguridad y cuentas diferenciadas de personas que administran el sistema.

En nuestra práctica reducimos este escenario a tres identidades para que se vea el principio:

- `adminlab` prepara datos y cuentas. No se utiliza como identidad normal de la aplicación.
- `lectorlab` puede realizar consultas sobre `curso_seguridad`, pero no escribir.
- `escritorlab` puede leer y modificar datos de esa base.

**Una cuenta que puede hacerlo todo hace imposible demostrar el mínimo privilegio.** Si conectamos todas las herramientas con `adminlab`, cualquier consulta funcionará, pero habremos aprendido poco sobre seguridad.

### Pregunta para comprobarlo

Una empresa necesita una pantalla que muestre ventas, pero nunca las modifique. ¿Qué ocurriría si usara un usuario con permiso para eliminar todas las bases? La pantalla podría funcionar normalmente; el problema está en las consecuencias de un fallo, una credencial robada o un uso indebido.

---

# Unidad 3. SCRAM: qué ocurre al identificarse con nombre de usuario y contraseña

## 3.1. No confundas una cuenta de MongoDB con un usuario del sistema operativo

MongoDB maneja sus propias identidades de base de datos. Una cuenta llamada `lectorlab` no tiene por qué existir como cuenta de Windows, macOS o Linux. El sistema operativo protege recursos del ordenador; MongoDB gestiona el acceso a sus operaciones y colecciones.

La instancia `mongo-seguridad` arranca con una cuenta administrativa definida a partir de variables del contenedor. Es una manera práctica de iniciar una base de pruebas desde cero. En una infraestructura empresarial, la provisión de credenciales y su custodia requieren mecanismos de secretos adecuados.

## 3.2. Qué significa SCRAM

SCRAM es una familia de mecanismos de autenticación basada en un intercambio de desafío y respuesta. La idea fundamental es que el cliente demuestra que conoce el secreto apropiado mediante un protocolo, en lugar de que el servidor deba confiar simplemente en que el nombre de usuario es verdadero.

No necesitas implementar SCRAM para comprender los ejemplos. Debes distinguir lo que hace:

- Ayuda a verificar **identidades mediante credenciales**.
- No determina por sí mismo si una cuenta puede insertar o borrar.
- No reemplaza la protección del canal de comunicación con TLS.
- No obliga a guardar contraseñas en texto claro dentro de scripts compartidos.

Una instrucción de conexión típica, lanzada en el contexto adecuado, tiene esta forma:

```
mongosh --username lectorlab --authenticationDatabase curso_seguridad --password
```

La opción `--password` sin escribir la contraseña en el comando solicita introducirla de forma interactiva. En el laboratorio la ejecutaremos dentro del contenedor, no suponiendo que todo el alumnado tiene `mongosh` instalado en el host.

## 3.3. Por qué importa authSource

MongoDB permite crear una identidad en una base de autenticación concreta. Esa base se especifica mediante `--authenticationDatabase` en `mongosh` o con `authSource` en una URI de conexión.

Ejemplo de dos datos que no hay que confundir:

- Datos que queremos leer: colección `secretos` de `curso_seguridad`.

- Base donde se reconoce la cuenta: también `curso_seguridad` para `lectorlab` en esta práctica, pero `admin` para `adminlab`.

Eso significa que una persona puede consultar una colección de una base mientras sus credenciales se autentican contra otra. **Base de datos de autenticación y base de datos consultada son conceptos diferentes**, aunque a veces coincidan.

Cuando una contraseña parece correcta pero el inicio de sesión falla, revisa la combinación de nombre, contraseña, servidor y base de autenticación. No cambies la cuenta a `root` como primera solución.

## 3.4. ¿Qué significa crear un usuario?

En `mongosh`, conectados con privilegios administrativos, podemos crear una identidad:

```
const curso = db.getSiblingDB("curso_seguridad");
curso.createUser({
  user: "lectorlab",
  pwd: process.env.LAB_READER_PASSWORD,
  roles: [{ role: "read", db: "curso_seguridad" }]
});
```

Cada elemento cumple una función:

- `user`: nombre de la identidad.
- `pwd`: credencial; `passwordPrompt()` pide introducirla sin dejarla escrita literalmente en el ejemplo.
- `roles`: lista de papeles autorizados.
- `role:"read"`: papel predefinido de lectura.
- `db:"curso_seguridad"`: ámbito en el que aplica ese rol.

Esta declaración no significa «el usuario está trabajando dentro de la colección». Significa que hemos registrado una cuenta de base de datos y asignado privilegios.

### Error previsible: «user already exists»

Si ejecutas dos veces la creación, quizá obtengas un error indicando que ya existe una cuenta con ese nombre. No es una anomalía misteriosa: el sistema ya tiene esa identidad. Puedes comprobarla con `curso.getUser("lectorlab")` mientras estás autorizado. Eliminarla o cambiar su contraseña es otra operación, que no debe realizarse sin saber si se está usando.

## 3.5. Credenciales fuera del repositorio

En ejemplos antiguos de cursos es frecuente ver comandos con `--password MiClave123`. Eso facilita copiar, pero puede exponer secretos en historiales de terminal, archivos compartidos o listas de procesos. En este curso evitamos incluir contraseñas reales. Los valores de `.env.example` son únicamente ejemplos y deben reemplazarse.

Un repositorio público puede contener instrucciones de instalación y nombres ficticios de usuarios. No debería publicar las claves privadas de certificados ni contraseñas reutilizables.

---

# Unidad 4. Roles, privilegios y RBAC con operaciones concretas

## 4.1. El permiso no se concede «para todo MongoDB»

En MongoDB los roles reúnen privilegios sobre recursos. Un recurso puede ser una base de datos, una colección u otro ámbito apropiado. Los privilegios describen acciones permitidas.

Para un principiante resulta útil empezar con dos roles predefinidos:

| Acción | `read` | `readWrite` |
|---|---|---|
| `find` sobre datos autorizados | Permitido | Permitido |
| `insertOne` | Denegado | Permitido |
| `updateOne` | Denegado | Permitido |
| `deleteOne` | Denegado | Permitido |

Esta tabla supone que hablamos de la base de datos a la que se concedió el rol; no afirma que esos nombres autoricen todas las colecciones de todos los servidores del mundo.

El resultado práctico no depende de si la operación la inicia una aplicación de C#, una consola `mongosh` o una herramienta gráfica. MongoDB aplica su política en el servidor.

## 4.2. Principio de mínimo privilegio

El mínimo privilegio no consiste en impedir trabajar a las personas. Consiste en otorgar **el permiso necesario para la tarea y no más**.

Si el departamento de informes solo necesita consultar ventas, asignarle `readWrite` puede parecer inocuo mientras nadie pulse «borrar». El riesgo aparece cuando una herramienta tiene un fallo, un script equivocado se ejecuta, o alguien consigue apropiarse de esa cuenta. El daño posible aumenta con sus privilegios.

Una buena pregunta de diseño es: «¿qué operaciones necesita realizar este proceso de verdad, y en qué colecciones?». No preguntes únicamente «¿con qué cuenta deja de dar error?».

## 4.3. Diferencias entre denegación y fallo técnico

Considera tres mensajes posibles:

1. No existe una conexión de red con el servidor.
2. El servidor rechaza la autenticación.
3. El usuario se autentica, pero la inserción obtiene una denegación de autorización.

Tienen causas diferentes. El primer caso puede deberse a un contenedor parado o a un puerto incorrecto; el segundo a credenciales o base de autenticación equivocadas; el tercero, a la definición de roles. La solución debe actuar sobre la causa real.

**No eleves privilegios para resolver problemas de red.** Tampoco reinicies bases de datos para resolver un filtro mal escrito.

## 4.4. Diseñar una prueba positiva y una negativa

Para demostrar que una política funciona no basta con comprobar que un administrador puede leer.

Una prueba convincente contiene:

- **Caso positivo:** `lectorlab` se identifica y lee un documento conocido.
- **Caso negativo:** el mismo lector intenta insertar y MongoDB lo deniega.
- **Caso de contraste:** `escritorlab` puede insertar porque su rol sí incluye escritura.

Con los tres resultados es posible demostrar que las identidades y los permisos importan. Una prueba que únicamente ejecutara todas las operaciones como `adminlab` no serviría para enseñar RBAC.

## 4.5. Riesgos del uso de roles personalizados

MongoDB permite construir roles más precisos, pero eso añade responsabilidad. Si defines un rol para leer exclusivamente una colección, debes comprobar la lista real de acciones y recursos concedidos. Un rol que parece inocente por su nombre puede contener privilegios adicionales.

Por eso, cuando estudiemos roles personalizados, no concluiremos que una cuenta es segura porque se llama `soloLectura`: inspeccionaremos qué permisos contiene.

### Actividad de diseño

Define tres tareas: consultar una factura, registrar una venta y administrar cuentas. ¿Por qué no deberían compartir la misma identidad? Propón un rol mínimo para cada una y explica qué operación negativa ensayarías para verificarlo.

---

# Unidad 5. TLS: proteger lo que viaja por la red

## 5.1. Una contraseña válida no protege por sí sola el canal

Un empleado puede identificarse correctamente y tener permisos adecuados, pero si cliente y servidor intercambian datos por una conexión insegura, existe otro tipo de riesgo. La pregunta ya no es «¿quién eres?» sino «¿puede un tercero interpretar o manipular la comunicación?».

**TLS** es un protocolo que protege la comunicación en tránsito y permite a un cliente verificar la identidad del servidor mediante certificados. Cuando el canal TLS se establece correctamente, los datos enviados por él quedan protegidos mediante mecanismos criptográficos adecuados para la conexión.

No es un mecanismo de cifrado de los archivos almacenados en disco. Si alguien copia un volumen donde están los datos, TLS no resuelve ese problema por sí solo.

## 5.2. ¿Qué aporta un certificado?

Un certificado digital vincula una identidad a una clave pública y contiene información para comprobar su validez y su emisor. Se utiliza junto con una clave privada asociada, que debe protegerse.

En una configuración básica:

1. El servidor MongoDB presenta su certificado.
2. El cliente comprueba, entre otras cosas, que confía en la autoridad que lo firmó y que el nombre del servidor coincide con los datos adecuados del certificado.
3. Si las comprobaciones son satisfactorias, el protocolo permite establecer un canal protegido.
4. Después siguen siendo relevantes la autenticación y la autorización necesarias para las operaciones.

La verificación del nombre importa: confiar en una autoridad no debería servir para aceptar indiscriminadamente cualquier servidor que presente cualquier certificado expedido por ella.

## 5.3. La función de una autoridad de certificación

Una **CA** o autoridad certificadora firma certificados. Para una demostración local podemos crear una CA temporal y firmar con ella un certificado de servidor y otro de cliente.

En el proyecto existe `scripts/generar-certificados.sh`, que prepara archivos de ejemplo dentro de `certs/`. Esa carpeta no se publica con claves privadas en Git. No debes copiar esa gestión simplificada de material criptográfico a una infraestructura de producción: allí se requieren procesos de custodia, renovación, revocación, permisos de archivos y confianza adecuados.

El nombre `server.pem` en un ejemplo puede incluir el certificado de servidor y su clave privada. El archivo `ca.crt` permite al cliente verificar la cadena de confianza. Confundir esos dos archivos puede llevar a errores de conexión.

## 5.4. «Requiere TLS» y «autenticado» no son lo mismo

La instancia del laboratorio `mongo-tls` está configurada con `--tlsMode requireTLS`. Esto significa que exige el protocolo TLS para conexiones de red a esa instancia.

Pero un cliente que consigue establecer un canal TLS todavía puede necesitar autenticarse para acceder a documentos protegidos. Por ejemplo, una orden de comprobación de disponibilidad `ping` puede responder sin demostrar que el cliente posee permisos para consultar datos.

De aquí surge una distinción que usaremos en las prácticas:

| Prueba | Qué puede demostrar | Qué NO demuestra |
|---|---|---|
| Conexión sin TLS rechazada | El servidor exige TLS | Que los usuarios tengan roles correctos |
| Conexión TLS y CA válida | El canal puede establecerse y el servidor es verificable | Que el cliente pueda leer secretos |
| Lectura autenticada permitida | La cuenta tiene permiso de lectura | Que también pueda modificar datos |
| Inserción denegada al lector | Se aplica una restricción de escritura | Que TLS está configurado correctamente |

No uses una sola prueba para afirmar que están verificadas todas las capas de seguridad.

## 5.5. Qué sucede si la conexión TLS falla

Los errores pueden aparecer por razones diferentes:

- No se pasó `--tls` aunque el servidor lo exige.
- El cliente no confía en la autoridad que firmó el certificado.
- El certificado está caducado o no es válido para el nombre solicitado.
- El archivo de clave/certificado es incorrecto o no puede leerse.
- La configuración de servidor y cliente no tiene opciones compatibles.

La forma pedagógica de resolverlo no consiste en añadir opciones de «no validar certificados» hasta que funcione. Eso desactiva precisamente una propiedad que queremos aprender a conservar.

---

# Unidad 6. X.509: identificar al cliente por su certificado

## 6.1. La misma palabra «certificado», dos usos

Hasta ahora el certificado servía principalmente para que el cliente verificara al servidor al establecer TLS. MongoDB también puede utilizar certificados de **cliente** para autenticar identidades mediante el mecanismo `MONGODB-X509`.

Son dos preguntas distintas:

- **¿Es legítimo este servidor?** El cliente verifica el certificado presentado por el servidor en la conexión TLS.
- **¿Quién es este cliente?** El servidor puede autenticarlo utilizando la identidad indicada por un certificado del cliente, cuando está configurado para ello.

No basta con tener un archivo `.pem`. Hay que entender si corresponde al servidor o al cliente, si está firmado por una CA de confianza, qué nombre representa y qué privilegios tiene esa identidad dentro de MongoDB.

## 6.2. Nombre distinguido y base $external

La identidad de un certificado X.509 se expresa mediante campos de un **nombre distinguido** (*Distinguished Name* o DN). En el laboratorio se utiliza un sujeto de demostración similar a:

```
CN=alumno-x509,OU=Alumnos,O=MongoDBFormacion
```

Ese sujeto no es una contraseña; identifica la entidad representada por el certificado. MongoDB utiliza la base especial `$external` para identidades autenticadas con mecanismos externos como X.509.

La cuenta de certificado necesita roles, igual que una cuenta basada en contraseña. Que el cliente demuestre poseer una clave privada válida no le concede automáticamente permiso para modificar cualquier colección.

## 6.3. Una prueba completa

Para demostrar X.509 de manera rigurosa, el laboratorio deberá verificar estas condiciones:

1. Se ha creado una CA local y un certificado de servidor válido para los nombres usados en la conexión.
2. La instancia se inicia exigiendo TLS.
3. Un cliente puede verificar la CA y establecer el canal.
4. Se registra el DN del alumno como usuario de `$external` con un rol de lectura sobre `curso_cert`.
5. El cliente se conecta con su certificado y el mecanismo `MONGODB-X509`.
6. La cuenta puede leer el documento de prueba.

7. La misma identidad recibe una denegación cuando intenta insertar.

La última prueba es importante: vuelve a demostrar que **autenticación y autorización están separadas**.

## 6.4. ¿X.509 está disponible en Community?

La autenticación X.509 de clientes puede demostrarse en MongoDB Community 8.0. Esto no significa que todas las funciones empresariales de identidad, auditoría y cifrado de MongoDB estén presentes en Community. Hay que verificar las capacidades concretas, no inferirlas por la mera presencia de TLS o certificados.

---

# Unidad 7. LDAP, cifrado en reposo y controles operativos

## 7.1. LDAP: qué necesidad atendía

En organizaciones grandes, gestionar cuentas individuales por separado en cada sistema puede resultar complejo. Los directorios centralizados permiten que distintas aplicaciones se integren con una infraestructura común de identidades y grupos.

LDAP es un protocolo utilizado históricamente en esos entornos. MongoDB ofrece capacidades de integración empresarial, pero en MongoDB 8.0 la integración LDAP está marcada como obsoleta. **No la vamos a representar como un laboratorio ejecutable sobre nuestra imagen Community.**

Para un alumno principiante, la lección no es aprender a configurar un servidor LDAP ficticio a base de comandos que fallarán. Es distinguir:

- Cuentas locales de una base MongoDB.
- Autenticación mediante un directorio corporativo.
- Autorización mediante roles que deben seguir definirse apropiadamente.

La integración con identidad corporativa puede facilitar administración centralizada, pero no elimina la necesidad de diseñar permisos de acceso a datos.

## 7.2. Cifrado en reposo: otra superficie de amenaza

Ya aprendimos que TLS protege datos en tránsito. ¿Qué ocurre cuando MongoDB escribe archivos en un disco o un volumen? Si una copia del almacenamiento cae en manos no autorizadas, podemos necesitar cifrado **en reposo**.

Hay mecanismos diferentes que no deben confundirse:

1. **Cifrado del sistema de archivos, disco o volumen:** depende del sistema operativo o proveedor de almacenamiento. Puede proteger bloques almacenados según su configuración y gestión de claves.
2. **Cifrado nativo en reposo de MongoDB autogestionado:** mecanismo propio de determinadas ediciones empresariales, con requisitos de configuración y gestión de claves.
3. **Cifrado a nivel de campo o aplicación:** puede atender necesidades específicas de datos sensibles y modelos de acceso; requiere un diseño diferente.

Un volumen Docker no es una garantía de cifrado. Que la aplicación conecte por TLS no demuestra que los archivos locales estén cifrados.

## 7.3. Protección de secretos

Las contraseñas y las claves privadas no deberían colocarse dentro del código de ejemplos destinado a publicarse. Para un laboratorio efímero se puede utilizar un archivo `.env` ignorado por Git y certificados temporales generados localmente. Esa solución simplifica la clase, pero no representa una política completa de secretos para producción.

Una empresa necesita decisiones sobre custodia, rotación, acceso, revocación y recuperación de claves. Un archivo de certificado válido pero mal protegido puede ser una vía de acceso no autorizado.

## 7.4. Copias de seguridad y restauración

La seguridad también incluye **disponibilidad e integridad**. Una copia de seguridad no sirve de mucho si nadie ha comprobado que puede restaurarla. Las copias deben protegerse frente a lectura y modificación no autorizadas, y deben encajar con los objetivos de recuperación de la organización.

La clase no realizará una auditoría de continuidad de negocio, pero debes comprender la diferencia entre:

- Tener una copia que alguien dice haber creado.
- Tener una copia verificada mediante una restauración.
- Tener procedimientos para recuperar el servicio y controlar quién puede acceder a las copias.

## 7.5. Registros y auditoría de operaciones

Los mensajes de `mongod` ayudan a investigar errores de inicio, fallos de conexión y otros sucesos operativos. No equivalen por sí solos a un sistema completo de auditoría empresarial que registre todas las acciones con el alcance exigido por una política regulatoria.

No confundas:

- Un log que dice «el servidor se inició».
- Una prueba que muestra que una operación fue denegada.
- Una política de auditoría que demuestra qué actores realizaron acciones sensibles a lo largo del tiempo.

La primera es un dato de operación, la segunda es una prueba de permisos y la tercera exige mecanismos, configuración y conservación de evidencias específicos.

## 7.6. Una arquitectura segura se defiende por capas

Para cerrar el capítulo, imagina una aplicación de ventas con dos cuentas: una para consultar y otra para registrar pedidos. La base contiene información personal y debe estar protegida ante conexiones externas no deseadas.

Una respuesta responsable incluiría:

- limitar el acceso a los puertos según necesidades reales;
- exigir identidad a las cuentas;
- aplicar mínimo privilegio a cada proceso;
- proteger comunicaciones con TLS;
- gestionar secretos y certificados con procedimientos adecuados;
- estudiar el cifrado del almacenamiento y las copias;
- supervisar errores y operaciones sensibles;

- comprobar restauraciones y actualizar el software cuando corresponda.

Ninguna medida sustituye por completo a las otras. La seguridad no es un único interruptor de «activado».

## Preguntas de cierre

1. Una conexión a través de TLS funciona, pero el cliente no puede insertar. ¿Qué control debes investigar?
2. Un administrador ha escrito una contraseña en un script publicado. ¿Qué riesgo existe aunque la base requiera autenticación?
3. ¿Por qué el cifrado de un volumen no equivale automáticamente a cifrar cada dato antes de enviarlo a la aplicación?
4. ¿Qué demostrarías con una prueba positiva y negativa de `lectorlab`?
5. ¿Por qué no sería correcto afirmar que la práctica ha validado LDAP o el cifrado nativo en reposo en Community?

La práctica avanzada comenzará comprobando TLS como canal, continuará con la identidad X.509 y terminará con una operación de lectura permitida y una escritura denegada. Los detalles empresariales se estudiarán como arquitectura, no como resultados inventados.

---

# Unidad 8 — Diseño de amenazas: cómo decidir qué proteger antes de escoger una opción de MongoDB

## Un sistema no es seguro solamente porque «se puede iniciar sesión»

Tenemos una pequeña aplicación de pedidos. Sus usuarios trabajan desde una oficina y, ocasionalmente, desde fuera de ella. Los empleados consultan pedidos, el servicio web registra otros nuevos y el equipo de soporte corrige incidencias. La base de datos contiene nombres y direcciones de entrega; la pérdida o alteración de esos datos tendría consecuencias.

Antes de configurar MongoDB es imprescindible determinar **qué daños queremos impedir**. Esta actividad se conoce como modelado de amenazas. No requiere ser especialista en criptografía: consiste en examinar actores, recursos, caminos de acceso y efectos de un incidente.

Distingamos cuatro ejemplos:

1. **Un visitante anónimo intenta leer pedidos.** Necesitamos cerrar el acceso desde redes no autorizadas y comprobar identidades y permisos.
2. **Una cuenta de informes intenta borrar una colección.** El problema no es que falte TLS; el problema es que una identidad puede tener más acciones de las necesarias.
3. **Alguien intercepta una conexión entre la aplicación y el servidor.** Debemos proteger y autenticar el canal mediante TLS correctamente configurado.
4. **Se pierde el disco y alguien recupera una copia del almacenamiento.** El control relevante es el cifrado en reposo y la custodia de sus claves, no la contraseña que introduce la aplicación en una conexión.

Si añadimos una contraseña administrativa compleja, pero todas las aplicaciones utilizan esa misma cuenta `root`, hemos mejorado solo una parte del sistema. Un programa comprometido podría actuar con todos los privilegios de esa cuenta.

## Construir una matriz de amenazas

Una matriz sencilla ayuda a no mezclar problemas:

| Recurso | Amenaza | Control candidato | Cómo comprobarlo |
|---|---|---|---|
| Colección de pedidos | Lectura anónima | Autenticación y red restringida | Intento anónimo rechazado |
| Colección de pedidos | Borrado con cuenta de lectura | RBAC con mínimo privilegio | `deleteOne` denegado |
| Comunicación cliente-servidor | Intercepción o suplantación | TLS y verificación de certificados | Conexión sin TLS rechazada; con CA válida |

| Recurso | Amenaza | Control candidato | Cómo comprobarlo |
|---|---|---|---|
| Claves y contraseñas | Publicación accidental | Gestión de secretos | Revisión de ficheros, permisos y repositorio |
| Copia de seguridad | Exposición de datos almacenados | Cifrado y acceso restringido | Inspección de configuración y restauración controlada |

La tabla separa **medidas propuestas** y **pruebas que podrían respaldarlas**. No confundas «he escrito TLS en el documento de arquitectura» con «TLS está realmente funcionando y se validan los certificados».

## Qué no vamos a simular en el laboratorio

El laboratorio se ejecuta con MongoDB Community 8.0 y contenedores efímeros. Podemos demostrar autenticación SCRAM, asignación de roles, denegaciones, TLS y X.509. No podemos declarar probado el cifrado nativo de MongoDB Enterprise en reposo ni una integración LDAP empresarial mediante unos comandos comunitarios que jamás podrían ejercer esas capacidades. Los temas se explican como arquitectura y requisitos de producto.

La evidencia debe respetar siempre el alcance real de la instalación.

---

# Unidad 9 — Usuarios, secretos y conexiones de aplicaciones

## No todas las cuentas deben vivir eternamente

En una aplicación, los servicios suelen necesitar credenciales para conectarse sin intervención humana. Eso crea una responsabilidad distinta de la del alumno que escribe una contraseña manualmente en `mongosh`. Las credenciales de la aplicación deben poder distribuirse de forma controlada, renovarse y retirarse si dejan de ser confiables.

Una cuenta de laboratorio llamada `escritorlab` representa una responsabilidad funcional, no necesariamente una persona. La aplicación que registra pedidos podría tener una identidad parecida, pero con permisos aún más acotados.

El ciclo de vida de una identidad comprende:

- **Alta:** se justifica la necesidad de acceso y se asignan permisos mínimos.
- **Uso:** las credenciales se entregan al componente autorizado y se protegen.
- **Revisión:** se comprueba si los permisos siguen siendo necesarios.
- **Rotación:** se reemplazan secretos cuando corresponde.
- **Retirada:** se impide continuar utilizando una identidad que ya no se necesita.

Crear cuentas es solo el primer paso. Una instalación que acumula usuarios antiguos con permisos excesivos puede seguir funcionando, pero no cumple una buena disciplina de seguridad.

## El problema de las contraseñas incrustadas

Considera un fragmento como este:

```
const contraseña = "MiClaveSuperSecreta";
```

Si se guarda en el repositorio, cualquiera con acceso al código puede verla. Si se copia en guías o capturas, la exposición se multiplica. La solución no consiste en cambiar el nombre de la variable a `passwordSegura`.

Para el laboratorio se utiliza un archivo `.env` local, excluido de Git, con valores que **deben sustituirse** por contraseñas de demostración. Este método simplifica la clase y evita publicarlas en el repositorio, pero **no es un gestor profesional de secretos**.

En producción, es necesario estudiar sistemas de custodia de secretos, identidades de servicios, permisos sobre archivos, procedimientos de recuperación y mecanismos de rotación que permitan cambiar credenciales sin interrumpir indebidamente la operación.

## URI de conexión: un formato cómodo, con riesgos

Una aplicación MongoDB puede utilizar una URI como:

```
mongodb://usuario:contraseña@servidor:27017/base?authSource=admin
```

Esta línea es **un esquema explicativo**, no un conjunto de credenciales válidas para pegar. La URI indica el protocolo, datos de autenticación, servidor, puerto, base y opciones. Si se escribe completa con una contraseña verdadera en un log, una captura o un commit, puede quedar expuesta.

También es importante el tratamiento correcto de caracteres especiales. Una contraseña que contiene símbolos reservados en las URI puede necesitar codificación adecuada. Una cadena de conexión mal formada puede ocasionar un fallo que parece de autenticación cuando realmente corresponde a cómo se representó el secreto.

En las prácticas preferimos pasar la contraseña a `mongosh` de forma interactiva o utilizar variables de entorno en procedimientos controlados; evitamos mostrar credenciales reales en el texto docente.

## La base authSource no concede permisos

Supongamos que la cuenta `adminlab` se autentica en la base `admin`. Eso no significa automáticamente que «el usuario es administrador» porque escribimos la palabra `admin` en `--authenticationDatabase`.

Lo que concede operaciones son **los roles efectivos asociados a esa identidad**, no el nombre de la base contra la que se comprueban las credenciales. Podemos tener una cuenta autenticada en `admin` con poderes muy limitados y otra autenticada en una base de aplicación con permisos ajustados a esa aplicación.

### Caso de análisis

Una aplicación utiliza una URI que conecta con `curso_seguridad` pero especifica `authSource=admin`. La cuenta `lectorlab` existe en `curso_seguridad`. El inicio de sesión falla.

Antes de cambiarle los roles, investiga la base donde se creó el usuario y la indicada para autenticar. **Añadir `root` no corrige una cuenta buscada en la base equivocada.**

---

# Unidad 10 — Cómo investigar incidentes sin destruir la evidencia

## El caso: «MongoDB no me deja entrar»

Un usuario informa que no puede ejecutar su consulta. La reacción impulsiva puede ser reiniciar Docker, borrar un volumen, cambiar contraseñas o conceder privilegios amplios. Todas esas acciones pueden ocultar o empeorar el problema.

Un diagnóstico ordenado empieza por separar capas.

**Primera capa: estado del servicio.** Comprobamos si el contenedor funciona y si `mongod` está preparado. `docker compose ps` y los logs ayudan a localizar fallos de inicio, rutas inexistentes, certificados caducados o problemas de recursos.

**Segunda capa: conexión.** Verificamos que el cliente utiliza el servidor y puerto correctos. La instancia `mongo-indices` no es la misma que `mongo-seguridad`, aunque ambas sean MongoDB.

**Tercera capa: autenticación.** Identificamos el usuario, el mecanismo y la base de autenticación. Una contraseña incorrecta se resuelve comprobando credenciales, no cambiando permisos de escritura.

**Cuarta capa: autorización.** Si la identidad funciona, pero una operación falla, revisamos roles y recursos. Es posible que sea una denegación deseada.

**Quinta capa: lógica de consulta.** Una consulta que devuelve cero documentos puede ser técnicamente válida. Tal vez el filtro no coincide con los datos, se consultó otra base o el identificador no existe.

Cada capa tiene una pregunta distinta. **No podemos diagnosticar las cinco con un solo comando `ping`.**

## Cómo escribir un informe útil

Un informe de fallo que diga «no funciona» obliga a otra persona a adivinarlo todo. Es mejor registrar:

- Acción que se intentó realizar, sin copiar contraseñas.
- Servicio y base de datos usados.
- Identidad o tipo de identidad empleado, sin secretos.
- Mensaje significativo del error.
- Resultado de una prueba de control, como una lectura permitida.
- Hipótesis razonada y siguiente comprobación propuesta.

No incluyas tokens, claves privadas, contraseñas, datos personales o volcados indiscriminados de logs en el informe.

## El papel de los registros y de la auditoría

Los logs operativos ayudan a saber qué ocurrió en el proceso servidor, pero **no son una garantía universal de trazabilidad de cada operación sobre datos**. Una política de auditoría formal requiere decidir qué acciones registrar, cómo conservar las evidencias, cómo protegerlas frente a manipulación y qué obligaciones normativas se aplican.

En MongoDB determinadas capacidades avanzadas de auditoría dependen de la edición y configuración. Nuestro laboratorio no pretende simular un sistema regulado de auditoría. Sí muestra cómo verificar denegaciones reales y cómo conservar una descripción legible de las pruebas.

## Un ejercicio completo de diagnóstico

Te entregan esta secuencia:

1. `docker compose ps` confirma que `mongo-seguridad` está saludable.
2. `adminlab` puede realizar `ping`.
3. `lectorlab` consigue leer `curso_seguridad.secretos`.
4. `lectorlab` recibe `Unauthorized` al ejecutar `insertOne`.

El mensaje final no indica que MongoDB se haya averiado. Demuestra que la identidad lectora carece de permiso de escritura, tal como esperábamos.

Ahora cambia el paso 3: la lectura también falla por autenticación. ¿Qué revisarías? El orden razonable es servidor, usuario, contraseña y `authSource`, antes de investigar reglas de escritura.

---

# Unidad 11 — TLS y certificados: errores que un principiante debe poder distinguir

## Un canal cifrado puede fallar por distintos motivos

Cuando activamos `requireTLS` en el servidor, la conexión del cliente debe negociar TLS. Pero no todas las situaciones que terminan en un error TLS tienen la misma causa.

**Caso A: cliente sin TLS.** El servidor exige canal cifrado; la negociación no puede realizarse como el cliente intenta. La solución es utilizar TLS correctamente, no desactivarlo en el servidor por comodidad.

**Caso B: CA desconocida.** El servidor presenta un certificado firmado por una autoridad que el cliente no reconoce como confiable. Debemos proporcionar la CA adecuada o corregir la instalación de confianza; no aceptar cualquier certificado sin verificar.

**Caso C: nombre del certificado no coincide.** El certificado es válido para un nombre, pero el cliente se conecta usando otro. La validación de identidad del servidor debe tener en cuenta los nombres declarados en el certificado, como los campos SAN correspondientes.

**Caso D: certificado caducado.** Un certificado tiene un intervalo de validez. Utilizarlo fuera de ese periodo debe tratarse mediante emisión y renovación apropiadas, no atrasando el reloj del equipo.

**Caso E: clave privada inaccesible.** El servidor puede conocer la ruta de `server.pem`, pero el proceso no tiene permisos para leerlo. Este es un problema de ficheros o configuración, no de los roles de las colecciones.

En el laboratorio tenemos una CA de demostración, certificados con validez muy corta y nombres preparados para localhost y el servicio. Sus permisos están simplificados únicamente para contenedores efímeros. **No deben copiarse a producción como procedimiento de custodia de claves.**

## La cadena de confianza no es la autorización

Una cuenta que presenta un certificado válido puede estar autenticada por X.509. Pero eso no concede automáticamente privilegios de lectura o escritura en todas las colecciones.

Piensa en un pasaporte: demostrar la identidad con un documento auténtico no significa tener permiso para entrar en todas las dependencias de un edificio. En MongoDB, una identidad X.509 tiene que estar asociada a roles adecuados.

El laboratorio comprueba una lectura permitida y una inserción denegada **con el mismo certificado de cliente**. Esa es una forma clara de separar autenticación y autorización.

## Un esquema mental de la conexión

```
CLIENTE                            SERVIDOR MONGODB
   |                                      |
   |---- solicita conexión TLS ------>|
   |<--- presenta certificado --------|
   |  verifica CA, nombre y validez   |
   |---- negocia canal protegido ---->|
   |                                      |
   |-- presenta identidad SCRAM/X509 ->|
   |<-- autenticación aceptada --------|
   |                                      |
   |--- solicita acción sobre datos -->|
   |<-- autorización permite/deniega --|
```

Este dibujo simplifica el protocolo real. Su finalidad no es describir cada mensaje criptográfico, sino mostrar que **formar el canal, demostrar identidad y conceder una operación son decisiones diferentes**.

## El error de «funciona con verificación desactivada»

Si una conexión solo se establece cuando el cliente desactiva la comprobación de certificados, **no podemos declarar la seguridad TLS correctamente verificada**. El canal puede estar cifrado, pero la identidad del servidor podría no haberse validado como exigía nuestra política.

En las prácticas buscamos que la conexión funcione **validando la CA**, no «sea como sea». Esa distinción es esencial para una persona que se inicia en la administración de bases de datos.

---

# Unidad 12 — Rúbrica final de decisiones de seguridad

## Un caso integrador

Una organización ficticia tiene tres procesos: lector de informes, registrador de pedidos y tarea de administración. Utiliza MongoDB y necesita proteger su base de pedidos.

Para el lector:

- Identidad independiente.
- Permiso de consulta sobre los recursos estrictamente necesarios.
- Prueba de lectura permitida.
- Prueba de inserción y borrado denegados.

Para el registrador:

- Identidad distinta.
- Privilegios de escritura sobre las colecciones adecuadas.
- Prueba de inserción permitida.
- Prueba de administración de usuarios denegada.

Para tareas administrativas:

- Identidad reservada.
- Uso controlado y supervisado.
- No emplearla como credencial habitual de una pantalla de usuario.

En cuanto a la conexión:

- Restringir la exposición de red.
- Proteger canales cuando corresponda con TLS, validando identidad del servidor.
- Evitar contraseñas y claves incrustadas en repositorios.
- Definir mecanismos de rotación y recuperación.

En cuanto al almacenamiento:

- Evaluar cifrado en reposo del sistema o edición apropiada.
- Proteger copias y comprobar restauraciones.
- No confundir funcionamiento con respaldo real de políticas empresariales.

## Qué entregaría un profesional

No entregaría únicamente una lista de diez comandos. Entregaría una **matriz de identidades y permisos**, una descripción de conexiones seguras, un listado de pruebas positivas y negativas y un diagnóstico de los controles no ensayados. La conclusión debe corresponder a la evidencia real.

**Una instalación de laboratorio que supera SCRAM, RBAC y X.509 no queda automáticamente certificada para producción.** Haber aprendido los mecanismos fundamentales permite formular mejores preguntas, no sustituye una evaluación de seguridad del sistema completo.

---

# Unidad 13 — La seguridad de los datos incluye recuperarlos después de un fallo

## 13.1. Un sistema puede estar protegido frente a intrusos y perder todos sus datos

Imagina un servidor que exige autenticación, utiliza TLS y tiene los roles correctamente definidos. Una persona autorizada comete un error y elimina una colección. O falla un disco. O un proceso defectuoso modifica muchos documentos.

La autenticación y el cifrado no restauran los datos por sí solos. Por eso en seguridad también hablamos de **disponibilidad**, **integridad** y **recuperación**, además de confidencialidad.

Estos tres términos pueden explicarse con ejemplos:

- **Confidencialidad:** alguien sin permisos no debe poder leer la dirección de un cliente.
- **Integridad:** una venta no debe modificarse silenciosamente por un actor no autorizado; sus datos deben conservar coherencia.
- **Disponibilidad:** la empresa necesita seguir accediendo a su información o recuperarla dentro de un plazo aceptable.

Un índice único puede proteger una regla de integridad, pero no sustituye una copia de seguridad. Un certificado TLS puede proteger el canal, pero tampoco puede reconstruir una colección eliminada.

## 13.2. Qué significa tener una copia válida

Una **copia de seguridad** es un conjunto de datos preservado para permitir recuperación. Pero «tener un archivo» no demuestra que podremos restaurarlo correctamente.

Para una base de datos en uso, hay que responder:

1. ¿Cuándo se tomó la copia y qué cambios posteriores quedarían fuera?
2. ¿Es coherente con las operaciones que estaban ocurriendo?
3. ¿Quién puede acceder a ella?
4. ¿Dónde se almacena y cómo se protege?
5. ¿Se ha ensayado alguna vez la restauración?
6. ¿Cuánto tiempo tardaríamos en recuperar el servicio?

Estos factores determinan si el plan de recuperación realmente sirve al negocio.

### RPO y RTO sin fórmulas innecesarias

**RPO** (*Recovery Point Objective*) expresa la cantidad máxima de datos recientes que una organización puede aceptar perder, medida normalmente como tiempo. Si se tolera una hora, la estrategia debe garantizar una recuperación compatible con ese requisito.

**RTO** (*Recovery Time Objective*) expresa cuánto tiempo se tolera hasta recuperar la operación. Una copia guardada en otro continente puede cumplir requisitos de protección ante determinados fallos, pero quizá no permita restaurar el servicio dentro del plazo esperado.

Los dos objetivos se relacionan, pero no son el mismo:

- RPO: **¿hasta qué momento podemos recuperar los datos?**
- RTO: **¿cuánto tardaremos en volver a funcionar?**

En una práctica introductoria no necesitamos diseñar una infraestructura completa de alta disponibilidad, pero sí reconocer por qué «la copia se creó ayer» es información insuficiente.

## 13.3. Restaurar no significa sobrescribir sin pensar

En producción, una restauración puede afectar a operaciones recientes y a otros servicios que utilizan la base. No se debe restaurar directamente sobre datos reales solo para comprobar si una copia funciona.

Una comprobación prudente utiliza un entorno aislado, datos de prueba y criterios de validación:

- se pueden abrir las colecciones esperadas;
- los documentos de control tienen valores conocidos;
- se han recuperado índices y estructuras necesarias o está documentado cómo reconstruirlos;
- las identidades y credenciales sensibles se gestionan correctamente;
- las aplicaciones pueden reanudar sus consultas sin introducir inconsistencias.

**No confundas una restauración técnica que termina sin error con la aceptación del sistema completo.** Las personas responsables necesitan saber si los datos restaurados son correctos para el negocio.

## 13.4. Seguridad de las copias

Una copia puede contener información tan sensible como la base original. Si alguien descarga un archivo completo sin restricciones, podría eludir la autenticación que protege al servidor activo.

Por eso se deben estudiar permisos sobre almacenamiento, cifrado, gestión de claves, acceso administrativo, retención y eliminación segura. No basta con poner `--auth` al servidor en funcionamiento si los archivos de respaldo quedan públicamente disponibles.

### Pregunta de aplicación

Una empresa exige que nadie pueda leer los pedidos sin autorización y tiene MongoDB correctamente configurado. Sin embargo, guarda copias sin proteger en una carpeta accesible a cualquier empleado.

¿Qué ha fallado? No necesariamente SCRAM. Ha fallado la protección del **otro lugar donde residen los mismos datos**.

---

# Unidad 14 — Separar redes, identidades y permisos en una arquitectura de aplicación

## 14.1. Tres componentes, tres responsabilidades

Imagina una aplicación web de ventas:

- Un navegador de cliente envía peticiones a un servidor web.
- El servidor web consulta y modifica datos en MongoDB.
- Una consola administrativa se utiliza solamente para tareas autorizadas de mantenimiento.

El navegador **no necesita conocer la contraseña de MongoDB** ni conectarse directamente al servidor de datos. La aplicación puede ofrecer funciones de negocio sin exponer sus credenciales internas a cada cliente.

Representación conceptual:

```
NAVEGADOR DEL CLIENTE
          |
          | peticiones de la aplicación
          v
SERVIDOR WEB
   |          |
   |          | identidad de servicio con permisos mínimos
   v          |
MONGODB <-----+
          ^
          |
CONSOLA ADMINISTRATIVA
   identidad distinta
```

Este dibujo simplifica las redes reales. Su propósito es demostrar que distintos actores necesitan distintos canales e identidades.

## 14.2. No compartir la cuenta administrativa

Una aplicación que utiliza siempre el usuario `adminlab` puede funcionar. Pero si una consulta está mal construida, una biblioteca vulnerable o una credencial acaba expuesta, el atacante potencialmente obtiene capacidades muy superiores a las necesarias.

La elección correcta comienza por enumerar operaciones:

- ¿Necesita leer?
- ¿Necesita insertar?
- ¿Necesita actualizar determinados campos?
- ¿Necesita borrar?
- ¿Necesita crear usuarios o roles? Probablemente no, salvo un servicio específico de administración.

A partir de ahí se asigna una identidad y el conjunto mínimo de acciones.

## 14.3. Una cuenta por responsabilidad facilita revocar

Supongamos que el servicio de importación deja de utilizarse. Si comparte credenciales con la aplicación principal, revocarlo puede romper la aplicación. Si tiene identidad propia, es posible retirar su acceso sin afectar a otros servicios.

La separación también facilita investigar errores: saber qué proceso ejecutó una acción suele ser más difícil cuando todos utilizan el mismo usuario administrativo.

No estamos afirmando que cada microoperación requiera una identidad diferente. La granularidad debe justificar el coste de gestionarla. El principio es **no compartir privilegios elevados sin necesidad**.

## 14.4. La red y los roles actúan en niveles distintos

Un cortafuegos o una configuración de red pueden impedir que una conexión llegue al puerto de MongoDB. El control de acceso de MongoDB decide qué puede hacer una identidad que sí ha llegado y se ha autenticado.

Son medidas complementarias. Un usuario de una red autorizada no debería recibir automáticamente permisos para escribir en todas las colecciones. A la inversa, una contraseña válida no debería hacer accesible un servidor a través de cualquier red por defecto.

En el laboratorio los puertos publicados están limitados a `127.0.0.1`. Es una elección adecuada para ejercicios locales, no una recomendación universal de arquitectura en producción. En una instalación real se definen redes privadas, reglas de acceso, identidad y monitorización según el despliegue.

## 14.5. El peligro de activar atajos durante pruebas

Hay atajos que facilitan una demostración y empeoran la seguridad:

- publicar MongoDB en todas las interfaces sin revisar quién alcanza el puerto;
- desactivar la validación de certificados TLS cuando falla la configuración;
- otorgar `root` a una aplicación lectora;
- guardar una contraseña en un repositorio para evitar introducirla;
- eliminar volúmenes para borrar un error sin comprender sus consecuencias.

Un buen laboratorio debe mostrar **cómo resolver correctamente** cada fallo. Si el alumno aprende que un error se soluciona desactivando controles, habrá aprendido una práctica dañina.

---

# Unidad 15 — Privilegios efectivos, usuarios heredados y prueba del mínimo privilegio

## 15.1. Los nombres de los roles no son pruebas de seguridad

Una cuenta puede llamarse `auditor`, `lector` o `visitante` y, sin embargo, tener más privilegios de los previstos. El nombre de la cuenta no concede ni restringe nada por sí solo.

Los privilegios efectivos dependen de los roles asignados, de los roles que esos roles puedan heredar y de los recursos sobre los que se autorizan acciones. Por eso un administrador debe inspeccionar las definiciones reales y no confiar en nombres amigables.

En los laboratorios comenzamos por `read` y `readWrite` porque son fáciles de comparar. Después se puede crear un rol propio que permita `find` únicamente sobre una colección específica.

## 15.2. Una cuenta puede acumular roles

Imagina una cuenta inicialmente autorizada a leer. Más adelante se le añade temporalmente un rol de escritura para corregir datos. Si nadie retira ese permiso, la cuenta conserva una capacidad superior a la necesidad original.

Es una forma de **deriva de privilegios**: los accesos se amplían con el tiempo, pero no se revisan ni eliminan.

Un proceso responsable contempla:

- justificar cada concesión;
- registrar por qué y durante cuánto tiempo se necesita;
- revisar los privilegios efectivos;
- retirar permisos al terminar la tarea;
- volver a probar una acción que deba ser denegada.

La comprobación negativa es tan importante como la positiva: si la cuenta ya no necesita escribir, el servidor debe rechazar su siguiente intento.

## 15.3. Una consulta no es un permiso de toda la base

Con un rol propio, podemos autorizar `find` sobre una colección y denegar esa misma acción sobre otra. Este es un nivel más preciso que `read` para toda una base.

Ejemplo conceptual:

```
privileges: [
  {
    resource: {
      db: "curso_seguridad",
      collection: "secretos"
    },
    actions: ["find"]
  }
]
```

La expresión indica explícitamente **dónde** y **qué acción** se permite. No significa que la cuenta tenga automáticamente todas las operaciones sobre `curso_seguridad` ni que pueda administrar otras bases.

## 15.4. ¿Cómo comprobarlo con evidencias?

Para ese rol:

1. Consulta un documento de `secretos` y comprueba que aparece.
2. Intenta insertar en `secretos` y comprueba que se deniega.
3. Intenta leer otra colección de la misma base y comprueba que se deniega, si el rol no la incluye.
4. Comprueba que la cuenta no tiene otros roles que amplíen accidentalmente esos permisos.

Una única operación permitida **no basta** para demostrar el límite. La política debe comprobarse con operaciones que la cruzan.

## 15.5. Identidad, sesión y cambios de privilegios

Cuando cambian los roles o las contraseñas, las conexiones que ya existen pueden presentar situaciones distintas de las nuevas sesiones según el mecanismo y momento de evaluación. Por eso las prácticas de modificación de roles vuelven a conectar antes de verificar el resultado.

No asumas que toda aplicación en producción refresca inmediatamente cada sesión cuando se retira un permiso. Define procedimientos y pruebas apropiadas para el despliegue concreto.

### Conclusión de la unidad

Una política de mínimo privilegio no se demuestra mostrando una tabla de roles ideal. Se demuestra con identidades concretas, recursos concretos, acciones permitidas y acciones rechazadas, y con un proceso para revisar los permisos cuando cambian las necesidades.